

Shells

Command-line interfaces, pipelines, and everyday text processing

tsaimh (2022-2026, CC BY-SA)

lctseng (2019-2021, CC BY-SA)

? (1996-2018)

What You Will Learn

- Explain what a shell does and how it differs from a terminal and the kernel.
- Configure startup files, variables, quoting, expansion, aliases, and job control.
- Use redirection and pipelines to connect small UNIX tools.
- Process text with grep, cut, awk, sort, uniq, tr, sed, and xargs safely.
- Recognize portability differences among sh/bash, tcsh, zsh, and PowerShell.

Course emphasis

Learn the portable ideas first. Shell-specific conveniences come second.

1. Foundations

What the shell is, what it is not, and how commands are executed.

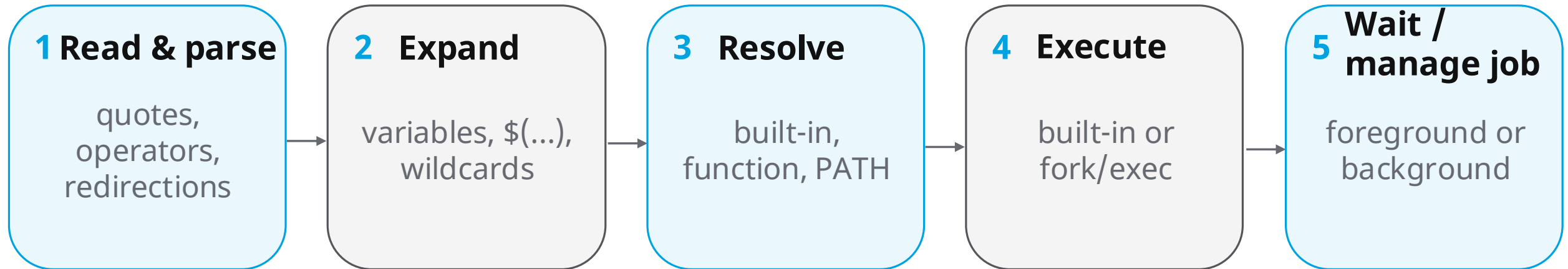
Shell, Terminal, and Kernel



```
$ pwd
/home/tsaimh
$ whoami
tsaimh
$ date
Sun Sep  7 20:00:00 CST 2026
```

- The terminal is a user interface; the shell is a program running inside it.
- The shell interprets syntax, expands variables and patterns, then runs built-ins or external programs.
- The kernel provides the system calls that programs ultimately use.

What Happens After You Press Enter in Your Shell?



```
$ ls "$HOME"/*.txt | grep report > matches.txt 2> errors.txt
```

Key idea

The shell processes syntax before most external programs ever see their arguments.

Common Shells

Shell	Family / role	Typical path	Notes
sh	POSIX-style Bourne shell	/bin/sh	Portable baseline for scripts
bash	Bourne Again Shell	/bin/bash	Common interactive shell on Linux
zsh	Bourne-style interactive shell	/bin/zsh	Default login shell on macOS
tcsh	C shell family	/bin/tcsh	Common in BSD/legacy environments
PowerShell	Object-oriented automation shell	pwsh / powershell.exe	Windows, Linux, and macOS

Portability rule

Do not assume “shell syntax” is universal. A script should declare the shell it targets.

Default Shells on Common Platforms

Defaults can be changed per user or by local policy.

```
> getent passwd tsaimh
tsaimh:*:12134:1010:蔡孟勳 Meng
Ts'ai:/u/faculty/tsaimh:/bin/tcsh
> echo $SHELL
/bin/tcsh
```

Platform	Typical default	Practical note
Debian	bash for new regular users	/bin/sh is a separate POSIX-style shell interface
FreeBSD 14+	sh for regular users and root	bash is not part of the base system
macOS	zsh	Terminal opens the user's configured login shell
Windows	PowerShell / cmd.exe	PowerShell 7 also runs on Linux and macOS

Check, do not guess
Use `getent passwd "$USER"` (Linux) or `getent/passwd` tools available on your system, and `echo "$SHELL"` as a hint.

Sources: Debian `adduser.conf(5)`, FreeBSD Quickstart Guide, Apple Terminal documentation, Microsoft Learn.

2. Startup Files

Understand when shell configuration files are read before editing them.

Login, Interactive, and Non-interactive Shells

Shell mode	Typical example	Why it matters
Interactive login	Console login; ssh session in some setups	Reads login startup files
Interactive non-login	New terminal tab; bash launched inside a shell	Reads interactive rc files
Non-interactive	Running a script: ./backup.sh	Usually skips interactive rc files

Common mistake

Putting required script variables only in ~/.bashrc can make a script behave differently under cron, SSH automation, or systemd.

Bash Startup Files

Mode	Files Bash reads
Interactive login	/etc/profile, then the first readable of ~/.bash_profile, ~/.bash_login, ~/.profile
Interactive non-login	~/.bashrc
Non-interactive	File named by \$BASH_ENV, if set
Login shell exit	~/.bash_logout

Typical pattern

A ~/.bash_profile often sources ~/.bashrc so interactive settings are shared.

GNU Bash Reference Manual, "Bash Startup Files".

FreeBSD sh and tcsh Startup Files

sh

- Login shell: ~/.profile
- Interactive shell: file named by \$ENV, if configured
- Keep portable settings in POSIX-compatible syntax

tcsh

- Reads system and user csh/tcsh startup files
- ~/.tcshrc or ~/.cshrc is used for interactive settings
- ~/.login and ~/.logout are login/logout specific

Course strategy

Use the shell you are given interactively, but write portable scripts in sh unless the assignment explicitly targets another shell.

See FreeBSD sh(1) and tcsh(1) manual pages for the exact startup order.

Editing Startup Files Safely

```
# ~/.profile or ~/.bashrc  
export EDITOR=vim  
export PATH="$HOME/bin:$PATH"  
alias ll="ls -lah"
```

- Read a file before copying or sourcing it.
- Make a backup before major edits.
- Apply changes with a new shell, or source the file explicitly.
- Keep system-wide and per-user settings separate.

Safer than curl | sh

Download, inspect, verify the source, then execute. Piping remote content directly into a shell bypasses that review step.

3. Variables, Quoting, and Expansion

Most shell surprises come from expansion rules, not from the command itself.

Shell Variables vs Environment Variables

Shell variable

```
name=alice  
echo "$name"
```

Visible to the current shell. Not automatically inherited by child processes.

Environment variable

```
export EDITOR=vim  
printenv EDITOR
```

Exported variables are inherited by programs started from the shell.

Remember

env / printenv show environment variables; set can show shell variables in many shells.

Useful Environment Variables

Variable	Meaning
HOME	User home directory
USER / LOGNAME	User name (availability varies by system)
SHELL	Configured login shell; not necessarily the currently running shell
PATH	Colon-separated directories searched for commands
PWD	Current working directory
EDITOR / VISUAL	Preferred text editor
LANG / LC_*	Locale and language settings

```
$ printf "%s\n" "$PATH"  
/usr/local/bin:/usr/bin:/bin
```

Assignment and Expansion

```
# POSIX-style shells
course=sa
month=$(date +%m)
export EDITOR=vim
printf "%s %s\n" "$course" "$month"
```

```
# tcsh
set course=sa
set month=`date +%m`
setenv EDITOR vim
echo "$course $month"
```

- In sh/bash, do not put spaces around = in a simple assignment.
- Quote expansions unless you intentionally want field splitting or pathname expansion.
- Prefer \$(command) over backticks in new sh/bash code.

Quoting Rules

Syntax	Meaning	Example
'text'	Literal text; no parameter or command substitution	echo '\$HOME'
"text"	Parameter and command substitution still occur	echo "\$HOME"
\	Escape the next character	echo *
unquoted	Field splitting and pathname expansion may occur	printf "%s\n" \$files

Be cautious!

Default habit

Use double quotes around variable expansions: "\$var".

Command Substitution

```
now=$(date)
count=$(wc -l < /etc/passwd)
printf "Now: %s\nUsers: %s\n" "$now" "$count"
```

- `$(command)` replaces the expression with the command's standard output.
- Trailing newlines are removed from the substituted output.
- The modern `$(...)` form nests cleanly and is easier to read than backticks.

```
# nesting
archive="$(basename "$(pwd)").tar.gz"
```

POSIX Shell Command Language supports both `$(...)` and backquotes; `$(...)` is preferred for new code.

Pathname Expansion (Globbing)

Pattern	Matches
*	Any string of characters, including an empty string
?	Any single character
[abc]	One character from the set
[!abc]	One character not in the set

```
$ ls test*  
test1 test2 test-5 testmess
```

```
$ ls test?  
test1 test2
```

```
$ ls test[12]  
test1 test2
```

Not the same as regex

Shell globbing is simpler than regular expressions. Quote a pattern when you want the program—not the shell—to interpret it.

Control Operators and Exit Status

```
$ false
$ echo $?
1
$ true
$ echo $?
0
```

Operator	Meaning	Example
;	Run the next command regardless of status	cd /tmp; pwd
&&	Run the next command only if the first succeeds	make && make test
	Run the next command only if the first fails	mkdir out exit 1

Convention

Exit status 0 means success. Non-zero means some kind of failure or special condition.

Escaping, Continuation, and Background Jobs

```
# Escape a literal *  
touch test\  
ls test\  
*
```

```
# Continue a long command  
printf "%s %s\n" \  
"hello" "world"
```

```
# Start in the background  
sleep 30 &
```

- \ protects a special character or continues a command on the next line.
- & starts a command as a background job in an interactive shell.
- Background does not mean detached: terminal signals and output can still matter.

For long-running services

Use a service manager such as systemd, rc.d, or a dedicated supervisor instead of relying on "&".

4. Built-ins and Job Control

Commands implemented by the shell can change the shell itself.

Built-in vs External Commands

Built-ins

- cd, export, alias, jobs, fg, bg, exit
- Run inside the shell process
- Can modify shell state

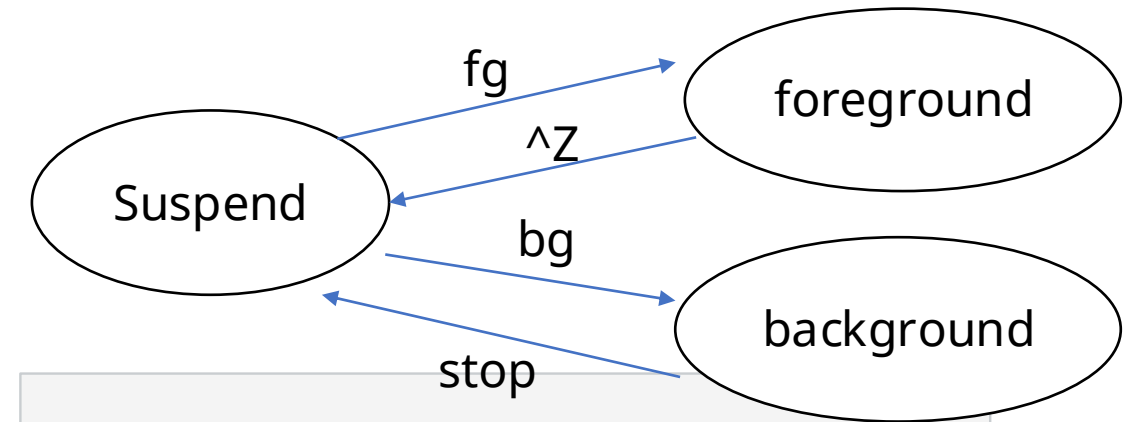
External commands

- ls, grep, awk, sort, ssh, python
- Found through PATH or an explicit path
- Run as separate processes

```
$ type cd
cd is a shell builtin
$ command -V grep
grep is /usr/bin/grep
```

Job Control

Action	Command / key
Start in background	command &
Suspend foreground job	Ctrl-Z
List jobs	jobs
Resume in background	bg %1
Bring to foreground	fg %1
Terminate	kill %1 or kill PID



```
$ sleep 300
^Z
[1]+  Stopped  sleep 300
$ bg %1
[1]+  sleep 300 &
$ jobs
[1]+  Running  sleep 300 &
$ fg %1
```

Signal model

Ctrl-Z sends a stop signal;
fg/bg change how the shell manages the
stopped/running job.

Aliases and the Directory Stack

```
[~] % dirs
~
[~] % pushd /var/log
/var/log ~
[/var/log] % pushd /etc
/etc /var/log ~
[/etc] % dirs
/etc /var/log ~
[/etc] % popd
/var/log ~
[/var/log] % popd
~
[~] %
```

```
alias ll="ls -lah"
alias ..="cd .."
```

- Aliases reduce typing in interactive shells; avoid depending on them in scripts.
- pushd / popd / dirs maintain a directory stack in bash, zsh, and tcsh.
- Functions are usually a better choice when an alias becomes complicated.

Caution

Aliases can hide destructive commands or change expected options. Know what an alias expands to before using it in administrative work.

5. I/O Redirection and Pipelines

UNIX tools become powerful when standard streams are connected deliberately.

Standard Streams and File Descriptors

FD	Name	Default destination/source
0	stdin	Keyboard / caller input
1	stdout	Terminal / caller output
2	stderr	Terminal / caller error stream



Redirection

Syntax	Effect
cmd < file	Use file as stdin
cmd > file	Write stdout to file; truncate first
cmd >> file	Append stdout to file
cmd 2> file	Write stderr to file
cmd >out 2>&1	Send stdout and stderr to the same file

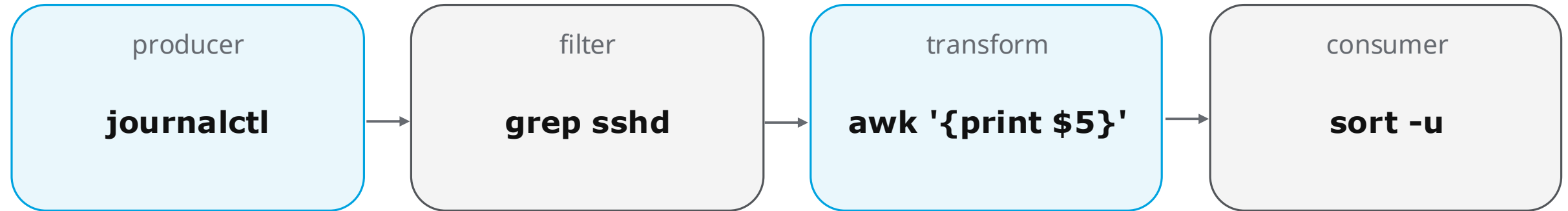
```
$ grep root /etc/passwd  
root:x:0:0:root:/root:/bin/bash
```

```
$ grep root /etc/passwd > result.txt  
$ grep nobody missing.txt 2> errors.txt  
$ make > build.log 2>&1
```

Order matters

2>&1 means “make fd 2 point where fd 1 points now.” In complex redirections, left-to-right order can change the result.

Pipelines



- A pipe (|) connects stdout of the left command to stdin of the right command.
- Each stage should do one small job well.
- stderr is not piped by | unless you redirect it explicitly.

Readable pipelines

Break long pipelines across lines and align stages so the data flow is obvious.

Managing stdout and stderr in Pipelines

```
# Keep output and errors separate  
command >output.log 2>error.log
```

```
# Merge before piping  
command 2>&1 | tee combined.log
```

```
# Bash shortcut (not POSIX sh)  
command |& tee combined.log
```

- Use separate files when errors need different handling.
- Merge streams only when you truly want them interleaved.
- tee copies a stream to both stdout and a file.

Portability

2>&1 is widely portable. |& is convenient in bash/zsh but is not POSIX sh syntax.

6. Text Processing Toolkit

Use small tools together instead of writing a new program for every one-off task.

File and Directory Commands

Command	Purpose
ls	List directory contents
pwd	Print working directory
cd	Change directory
mkdir	Create directories
rmdir	Remove empty directories
cp	Copy files/directories
mv	Move or rename
rm	Remove files/directories
ln	Create hard or symbolic links
stat	Show file metadata

Destructive commands

rm has no universal recycle bin. Double-check paths, quoting, and wildcards before running recursive or privileged removal commands.

```
$ pwd
/home/tsaimh
$ mkdir -p lab/logs
$ cp config.example
lab/config
$ stat lab/config
```

Text Processing: A Mental Map

Question	Tool
Need the first/last lines?	head / tail
Need lines matching a pattern?	grep
Need selected fields or columns?	cut / awk
Need ordering or grouping?	sort / uniq
Need counts?	wc / uniq -c
Need character translation/deletion?	tr
Need stream edits?	sed
Need to convert input into command arguments?	xargs

head, tail, and grep

```
# First / last lines
head -n 20 file.txt
tail -n 50 file.txt

# Follow a growing log
tail -f /var/log/messages

# Search with line numbers
grep -n "sshd" file.txt

# List matching file names
grep -l "ERROR" *.log
```

- Use -F for fixed strings when you do not need regular expressions.
- Use -E for extended regular expressions when appropriate.
- Quote patterns to prevent the shell from expanding special characters first.
- On systemd systems, journalctl often replaces direct log-file reading for many services.

cut and awk: Selecting Fields

```
# /etc/passwd: user, uid, home, shell  
$ grep '^tsaimh:' /etc/passwd | cut -d: -f1,3,6,7  
tsaimh:1001:/home/tsaimh:/bin/tcsh
```

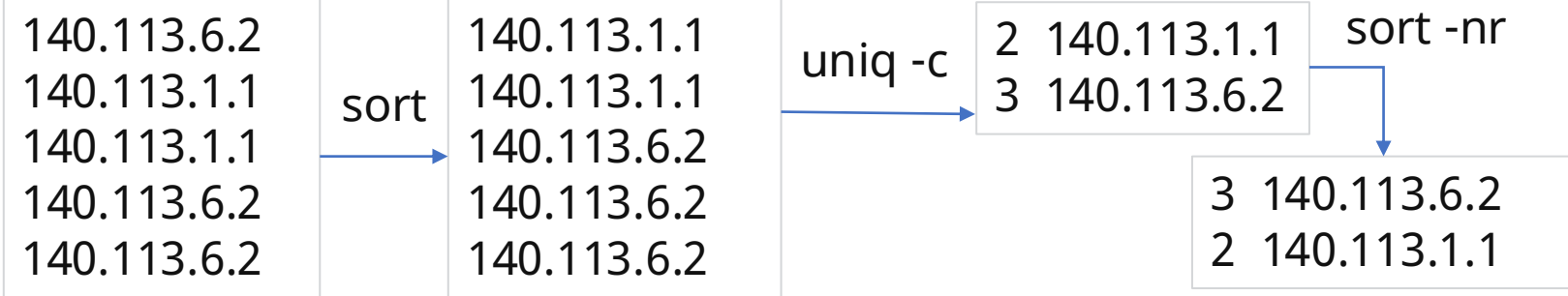
```
# Same idea with awk  
awk -F: '$1=="tsaimh" {print $1, $3, $6, $7}' /etc/passwd
```

Tool	Best for	Limitation
cut	Simple fixed delimiters or character ranges	No conditions or calculations
awk	Field-aware filtering, formatting, calculations	More syntax to learn

Avoid parsing ls in scripts

ls output is for humans and can be ambiguous. Prefer tools that expose structured file information when scripting.

sort, uniq, and wc



```
# Sort numerically by the 5th whitespace-separated field
sort -n -k 5,5 file.txt
```

```
# Count repeated lines, list from most to least
$ sort ip.txt | uniq -c | sort -nr
3 140.113.1.1
2 140.113.6.2
```

```
# Count lines, words, bytes
wc -l file.txt
wc -w file.txt
wc -c file.txt
```

- `sort -n`: numeric order;
• `-r`: reverse;
• `-u`: unique;
• `-k`: key field;
• `-t`: delimiter.
- `uniq` only detects adjacent duplicate lines, so `sort` usually comes first.
- `wc` is useful both alone and at the end of a pipeline.

String vs numeric

Without `-n`, "14" may sort before "123" because text comparison is not numeric comparison.

tr and sed

```
# Translate lowercase to uppercase
printf "%s\n" "Hello World" | tr '[:lower:]' '[:upper:]'

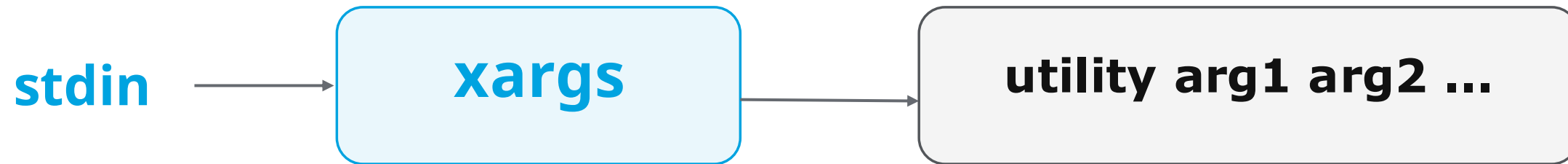
# Delete tabs
tr -d '\t' < file.txt

# Squeeze repeated spaces
tr -s ' ' < file.txt

# Replace the first occurrence per line
sed 's/old/new/' file.txt
```

- tr works character by character; it does not understand words or regular-expression capture groups.
- sed edits streams line by line and supports regular-expression-based substitutions.
- Use locale-aware character classes such as `[:lower:]` when portability matters.

xargs: Turn Input into Arguments



```
$ printf "%s\n" a b c | xargs echo  
a b c
```

```
$ printf "%s\n" a b c | xargs -n1 echo  
a  
b  
c
```

Use case

`xargs` is useful when a program expects command-line arguments instead of reading `stdin`.

xargs Options: Portable vs FreeBSD-specific

Option	Availability	Meaning
-n N	POSIX / common	Use at most N input arguments per invocation
-I token	POSIX / common	Replace token with one input line
-s size	POSIX / common	Limit command-line size
-J token	FreeBSD extension	Insert input at the first token instead of appending

```
$ ls | xargs -I % echo "[%]"  
[f1.txt]  
[f2.png]  
# FreeBSD only  
$ ls | xargs -J % echo % here %  
f1.txt f2.png here %
```

POSIX specifies -I and -n; FreeBSD xargs(1) additionally documents -J.

Safe xargs with Filenames

```
# Unsafe for filenames containing  
whitespace/newlines  
find . -type f | xargs grep -l TODO
```

```
# Robust: NUL-delimited records  
find . -type f -print0 | xargs -0 grep -l TODO
```

- Normal xargs input is separated by blanks and newlines.
- Filenames can legally contain spaces, tabs, and even newlines.
- Use `find -print0` with `xargs -0` when processing arbitrary filenames.

Even simpler sometimes

Many find implementations support `-exec`, which can avoid xargs entirely:
`find ... -exec command {} +`

Integrated Example: Explore /etc/passwd

```
# 1) Show login name and shell
cut -d: -f1,7 /etc/passwd | head

# 2) Count how many accounts use each shell
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -nr

# 3) Show accounts whose shell is not nologin
awk -F: '$7 !~ /nologin$/ {print $1, $7}' /etc/passwd
```

Think in stages

- Choose fields
- Filter records
- Transform output
- Sort / aggregate
- Format for humans or another program

Pipeline design

Each stage should have a clear input and output contract.

Integrated Example: Summarize a Log

```
# Suppose access.log begins with  
client IP in field 1  
awk '{print $1}' access.log \  
| sort \  
| uniq -c \  
| sort -nr \  
| head -n 10
```

Top client IP addresses

- awk extracts the field.
- sort groups identical values.
- uniq -c counts adjacent duplicates.
- sort -nr ranks counts numerically in reverse order.
- head keeps the top 10.

This is the UNIX philosophy in practice

Compose simple tools instead of writing a one-off parser.

The UNIX Philosophy

Small tools. Clear interfaces. Composition.

Do one thing well

Prefer focused tools with predictable behavior.

Text as glue

Pipes make it easy to connect independent programs.

Automate repetition

Turn a successful command sequence into a script.

Modern interpretation

Use composition when it improves clarity. Do not force text pipelines where structured APIs, databases, or a real programming language are safer.

Reference: https://en.wikipedia.org/wiki/Unix_philosophy

ShellCheck: Static Analysis for Shell Scripts

```
#!/bin/sh
for f in $files; do
  rm $f
done
```

What ShellCheck catches

Unquoted expansions, portability issues, suspicious tests, unused variables, common logic mistakes, and more.

```
$ shellcheck backup.sh
```

In backup.sh line 2:

```
for f in $files; do
```

 ^-----^ SC2086: Double quote to
prevent globbing and word splitting.

- Run it before committing scripts.
- Treat warnings as prompts to understand the shell semantics, not as a substitute for testing.

ShellCheck: shellcheck.net / github.com/koalaman/shellcheck

Shell Cheat Sheet

Task	Pattern
Variable	name=value ; export name=value
Safe expansion	"\$var" ; "\${var}"
Command substitution	\$(command)
Conditional sequence	cmd1 && cmd2 ; cmd1 fallback
Redirect	> >> < 2> 2>&1
Pipe	producer filter consumer
Jobs	jobs bg %1 fg %1
Find + xargs safely	find ... -print0 xargs -0 ...

Most valuable habit

Quote variable expansions, test incrementally, and inspect commands before adding sudo.

Appendix

History shortcuts and reference material.

Command History: Everyday Recall

Action	bash/zsh/tcsh
Show history	history
Previous command	Up arrow
Reverse search	Ctrl-R (bash/zsh; tcsh behavior/configuration differs)
Re-run command by number	!n
Re-run previous command	!!
Re-run most recent command starting with text	!text

Be careful with ! history expansion

Always inspect what will run. History substitutions can surprise you in destructive or privileged commands.

tcsh History Substitution

Syntax	Meaning
!n	Execute history entry n
!-n	Execute the command n entries back
!!	Previous command
!str	Most recent command beginning with str
!?str	Most recent command containing str
!!:*	All arguments of the previous command
!!:s/old/new/	Substitute text in the previous command

```
% history
15 cd /etc
16 ls HOSTS FSTAB
17 history
% cat !-
2:*:s/HOSTS/hosts/:s/FSTAB/fstab
cat hosts fstab
```

Modern practice

Interactive history shortcuts are useful, but scripts should use explicit commands and variables.

References

- GNU Bash Reference Manual — <https://www.gnu.org/software/bash/manual/>
- POSIX Shell Command Language — https://pubs.opengroup.org/onlinepubs/9799919799/utilities/V3_chap02.html
- FreeBSD Quickstart Guide for Linux Users — <https://docs.freebsd.org/en/articles/linux-users/>
- FreeBSD sh(1), tcsh(1), xargs(1) — <https://man.freebsd.org/>
- Debian Reference / adduser.conf(5) — <https://www.debian.org/doc/manuals/debian-reference/> and Debian Manpages
- Apple Terminal User Guide — <https://support.apple.com/guide/terminal/>
- Microsoft Learn: What is PowerShell? — <https://learn.microsoft.com/powershell/scripting/overview>
- ShellCheck — <https://www.shellcheck.net/> ; <https://github.com/koalaman/shellcheck>

Manual pages first

For commands on a specific machine, the commands' man pages and the platform documentation are the authoritative references.